ClientsMine Backend Handbook

A complete interactive guide to the architecture, flow, and code of the ClientsMine backend server.

Welcome to the ClientsMine Backend Handbook. This guide was generated to help developers understand how the server-side scraper and API modules interact, how data flows through the agentic pipeline, and what each individual file is responsible for.

System Flow & Architecture

ClientsMine utilizes an event-driven, streaming architecture. Instead of waiting for a long batch process to finish before showing results, the server streams leads to the client progressively using **Server-Sent Events (SSE)**.

1. Client Request (HTTP POST /api/ai/client)

User submits search criteria. Server opens an SSE connection and holds it open.

2. Intent Analysis (PhaseOne / intentParser.js)

Gemini Flash parses the query into structured parameters (role, category, timeframe).

3. Parallel Scrape (streamingOrchestrator.js)

Selected sites are scraped in parallel. Whichever site finishes first is parsed instantly.

4. Extraction & Optimization (dataCollector.js & dataOptimizer.js)

Leads are extracted (using custom Cheerio parsers or LLM) and checked for duplicate/date constraints.

5. Progressive SSE Push (aiClientAnalyzer.js)

Optimized leads are pushed to the client immediately. Stream closes once all sites complete.

Configs Folder

Contains configuration logic for database and external connections.

db.js

Purpose: Connects the application to MongoDB via Mongoose.

Interactions: Imported by `server.js` at startup.

configs/db.js

```
const mongoose = require('mongoose')

const connectDB = async () =>{
    try {
        await mongoose.connect(process.env.MONGO_URI)
        console.log("Database connected successfully")
    } catch (error) {
        console.log("Database connection failed", error)
        process.exit(1)
    }
}

module.exports = connectDB
```

Explanation: Calls `mongoose.connect()` using the connection string from environment variables. If the connection fails, it catches the error and calls `process.exit(1)` to shut down the server immediately rather than running in an unstable, disconnected state.

Models Folder

Defines the schemas and structures of the MongoDB collections.

user.js

Purpose: Stores user profiles, hashed credentials, and billing tiers.

models/user.js

```
const mongoose = require('mongoose')

const userDetails = new mongoose.Schema(
    {
        username: { type: String, required: true },
        email: { type: String, required: true, unique: true },
        password: { type: String, required: true },
        country: { type: String, required: true },
        state: { type: String, required: true },
        city: { type: String, required: true },
        phone: { type: String, required: true, unique: true },
        occupation: { type: String, required: true },
        isPro: { type: Boolean, default: false },
    }, { timestamps: true }
)

module.exports = mongoose.model("User", userDetails)
```

Explanation: Defines the user fields. The `isPro` field controls access limits (free accounts are capped at 5 leads per search).

usersMessage.js

Purpose: Logs search queries and tracks task progress.

models/usersMessage.js

```
const mongoose = require('mongoose')

const UserMessageSchema = new mongoose.Schema(
    {
        userId: { type: String, required: true, ref: "User" },
        title: { type: String, required: true },
        message: { type: String, required: true },
        status: {
            type: String,
            enum: ['PROCESSING', 'COMPLETED', 'ERROR', 'CLARIFICATION_REQUIRED'],
            default: 'PROCESSING'
        },
        error_details: { type: String }
    }, { timestamps: true }
)

module.exports = mongoose.model("UsersMessage", UserMessageSchema)
```

Explanation: Links queries to users. Maintains an enum field to track if a query is still running or has completed/failed.

scrapedLead.js

Purpose: Stores individual extracted leads.

models/scrapedLead.js

```
const mongoose = require('mongoose');

const scrapedLeadSchema = new mongoose.Schema({
    userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
    userMessageId: { type: mongoose.Schema.Types.ObjectId, ref: 'UsersMessage',
required: false },
    domain_category: { type: String, required: true, index: true },
    company_name: { type: String, default: 'NA' },
    job_title: { type: String, required: true },
    city: { type: String, default: 'NA' },
    country: { type: String, default: 'NA' },
    posted_date: { type: String, default: 'NA', index: true },
    email: { type: String, default: 'NA' },
    phone: { type: String, default: 'NA' },
    website: { type: String, default: 'NA' },
    linkedin: { type: String, default: 'NA' },
    source_url: { type: String, required: true },
    scraped_at_utc: { type: Date, default: Date.now },
    tier: { type: Number, required: true, index: true }
}, { timestamps: true });

scrapedLeadSchema.index({ userId: 1, tier: 1, createdAt: -1 });

module.exports = mongoose.model('ScrapedLead', scrapedLeadSchema);
```

Explanation: Contains the properties of each lead. A compound index on `userId, tier, createdAt` ensures the dashboard loads leads sorted by quality (tier 1: has email) instantly.

Middlewares Folder

auth.js

Purpose: Route guard that checks if a request contains a valid JWT.

middlewares/auth.js

```
const jwt = require('jsonwebtoken');
const User = require('../models/user');

const protect = async (req, res, next) => {
    let token;
    if (req.headers.authorization &&
req.headers.authorization.startsWith('Bearer')) {
        token = req.headers.authorization.split(' ')[1];
    }
    if (!token) {
        return res.status(401).json({ message: "Not authorized" });
    }
    try {
        const decoded = jwt.verify(token, process.env.JWT_SECRET);
        req.user = await User.findById(decoded.userId).select('-password');
        next();
    } catch (error) {
        return res.status(401).json({ message: "Token failed" });
    }
};

module.exports = protect;
```

Explanation: Extracts the bearer token, verifies it, and attaches the user document (minus password) to the request object before calling `next()`.

Utilities Folder

generateToken.js

Purpose: Helper for session tokens.

utils/generateToken.js

```
const jwt = require('jsonwebtoken')

const generateToken = (res, userId) => {
    const accessToken = jwt.sign({ userId }, process.env.JWT_SECRET, { expiresIn:
"15m" })
    const refreshToken = jwt.sign({ userId }, process.env.JWT_SECRET, { expiresIn:
"7d" })

    res.cookie('refreshToken', refreshToken, {
        httpOnly: true,
        secure: process.env.NODE_ENV === 'production',
        sameSite: 'strict',
        maxAge: 7 * 24 * 60 * 60 * 1000,
    })

    return accessToken
}
```

Explanation: Returns a short-lived JSON access token and sets a secure long-lived cookie for seamless session refresh.

antiBotScraper.js

Purpose: Network requests and browser automation.

Explanation: Evaluates domain rate-limits (max 1 request/3s per domain) and chooses between three request flows:
1. `fetchApiJson`: Fast 200–500ms delay API fetcher.
2. `fetchStaticHtml`: Axios scraper with exponential backoff on 429 status codes.
3. `fetchDynamicHtml`: Stealth Puppeteer scraper for sites requiring client-side rendering.

Routers Folder

Exposes and maps URL endpoints to controllers.

Router	Prefix	Endpoints
`authRoutes.js`	`/api/auth`	`/signup`, `/login`, `/me`, `/upgrade`, `/refresh-token`, `/logout`
`clientRoutes.js`	`/api/ai`	`POST /client` (SSE), `GET /status/:messageId`
`leadRoutes.js`	`/api/leads`	`GET /` (paginated list), `GET /history`

Controllers Folder

Processes requests and sends back responses.

aiClientAnalyzer.js

Purpose: Starts the scraping pipeline and streams progress via Server-Sent Events.

Code logic:
- Creates a new `UsersMessage` document in `PROCESSING` status.
- Configures the response headers to `text/event-stream`.
- Triggers the parallel orchestrator. When a site completes, the callback filters results (e.g. enforcing the 5–lead limit for free accounts) and calls `res.write()` to push data instantly.

leadController.js

Purpose: Querying leads database.

Code logic:
- `getLeads`: Implements paginated retrieval using Mongoose `.skip()` and `.limit()`.
- `getHistory`: Groups user queries with their associated results by performing a lookup on `UsersMessage`, then retrieving related `ScrapedLead` documents.

Scraping Pipeline (Implementations/)

The backend scraping engine is divided into independent processing nodes.

streamingOrchestrator.js

Purpose: Drives the parallel scraping workflow.

Explanation: Maps each selected site to an asynchronous promise chain: `scraperAgentNode -> dataCollectorNode -> optimizeAndSaveLeads -> onLeads()`. All promises are executed concurrently using `Promise.allSettled()`, ensuring that slow or blocked sites do not delay the streaming of faster sites.

siteSelector.js

Purpose: Selects target job boards and formats search URLs.

Explanation: Evaluates the query's category and maps it to target sites (LinkedIn, RemoteOK, WeWorkRemotely, HackerNews). Builds search URLs by constructing slugs or query parameters.

scraperAgent.js

Purpose: Handles the raw extraction of page content.

Explanation: Routing logic:
- HackerNews API: Uses the fast JSON path.
- Other sites: Scrapes page source, loads the content into Cheerio, strips non-text tags (like styles, scripts, navigation, footers), and returns clean, condensed HTML.

dataCollector.js

Purpose: Extracts structured lead objects from raw page content.

Explanation: Utilizes hybrid extraction:
- Direct JS/Cheerio parsers: Parses structured sources (LinkedIn, HackerNews, RemoteOK) instantly in JS, bypassing the LLM.
- Cheerio pre-extraction: For unstructured sites like WeWorkRemotely, extracts job card text into a compact block, reducing the text size sent to Gemini by 99%.

Optimization Folder

dataOptimizer.js

Purpose: Cleans, filters, and saves leads.

Core logic:
1. **Timeframe Filter**: Parses relative/absolute dates. Stale leads (older than 2 months) are discarded.
2. **Deduplication**: Queries the database to prevent duplicate entries for the same company and job title.
3. **Quality Tiering**: Categorizes leads by contact availability. Leads with email addresses are assigned to Tier 1.
4. **Sort**: Sorts leads to place Tier 1 entries at the beginning of the array before saving them to the database and streaming them to the client.